queryX: An Explainable SQL Compiler with Bytecode Generation and a Self-Verifying Virtual Machine

Yashasvi Dagar
Reg. No. 24BCI0112

Kshitij Rahul Manikshete
Reg. No. 24BCI0080

Deepanshu
Reg. No. 24BDS0179

Compiler Design Lab Project Report

***Abstract*—Query optimizers make decisive choices about join order, join algorithm and access path, yet expose only a final plan, while compiler courses teach lexing, parsing, intermediate representations and code generation as separate topics. This report presents queryX, a compiler for a subset of SQL, built from first principles, that exposes every phase from tokens to executed code. A hand-written lexer and recursive-descent parser feed a semantic analyzer and a relational-algebra intermediate representation, which rule-based rewrites and a cost-based planner turn into a physical plan. Unlike optimizer visualizers that stop at the plan, queryX compiles the selected plan into bytecode for its own stack-based virtual machine using the produce/consume model, improves the bytecode with a peephole optimizer, and executes it. Every compilation verifies itself: all candidate plans are compiled and executed and must return identical rows. A grammar- and schema-aware random query generator fuzzes the pipeline; on 200 random queries, all 936 executed plans agreed, and fuzzing exposed a planner defect involving filters above outer joins. Peephole optimization removed 5.5% of executed instructions on average and up to 26.2%.**

***Index Terms*—compiler design, query compilation, code generation, peephole optimization, virtual machine, differential testing, fuzzing.**

I. INTRODUCTION

SQL is declarative: a query states *what* result is wanted, never *how* to compute it. A database system bridges that gap with a query optimizer, which chooses join orders, join algorithms and access paths using cost-based optimization [1]. The result is visible only as a final plan, usually through an `EXPLAIN` command. Compiler courses, meanwhile, teach lexical analysis, parsing, intermediate representations, optimization and code generation [6] as separate topics, and students rarely see one system that connects them.

The objectives of this project are to:

- build the compiler front end by hand: lexer, recursive-descent parser, abstract syntax tree (AST) and semantic analysis;
- lower SQL to a relational-algebra intermediate representation (IR) and optimize it with explainable rules;
- enumerate and cost physical plans and select the cheapest;
- generate target code for the selected plan, optimize it, and execute it on a virtual machine (VM);
- verify the optimizer automatically and visualize every stage in a web dashboard.

The main contributions are an end-to-end, inspectable query compiler with a bytecode back end; self-verification of every compilation by differential testing [7] of candidate plans; and a random query generator that fuzzes the whole pipeline. Section II surveys related work, Section III presents the proposed system, Section IV describes the implementation, Section V reports results, and Section VI concludes.

II. LITERATURE SURVEY

Cost-based optimization. Selinger et al. [1] introduced cost-based access path selection and dynamic programming over join orders for System R. Nearly every relational optimizer follows this design, and queryX uses it for left-deep join trees.

Query execution. The Volcano model [4] executes a plan as a tree of iterators, calling each operator once per row. Neumann [2] showed that compiling a plan into tight loops, by letting each operator *produce* rows and *consume* its child's rows inside generated code, is far more efficient; HyPer and Umbra [3] are built on this approach. These systems generate machine code and do not expose it to users.

Code optimization. McKeeman's peephole optimization [5] improves code by rewriting short instruction windows, and remains a standard technique in compiler back ends [6].

Testing. Differential testing [7] detects bugs by comparing implementations that should agree. Query synthesis is now widely used to test database engines [8].

Research gap. Existing tools either expose only the final plan or focus on one stage. None of the surveyed systems combines a visible, hand-built front end with a visible code-generation back end and built-in verification. queryX addresses this gap.

III. PROPOSED SYSTEM

queryX is split into a stateless FastAPI backend that owns the compiler pipeline and a React dashboard that shows each stage. Fig. 1 shows the pipeline. The front end and optimizer follow the classic design; the back end, shown dark, is the novel part of this work. No database is needed: generated code runs on the built-in VM against sample data generated from the optimizer's statistics.

Front end and optimizer (standard):

Lexer	Parser	Semantic
IR builder	Rule optimizer	Planner + cost

Back end (proposed):

Code generator	**Peephole optimizer**	**Virtual machine**	**Plan verification**

Fig. 1. The queryX pipeline: SQL source in, executed and verified result rows out.

TABLE I
Compiler Phases in queryX

Phase	queryX component	Output
Lexical analysis	Hand-written lexer	Tokens with positions
Syntax analysis	Recursive-descent parser	AST
Semantic analysis	Catalog as symbol table	Coded diagnostics
Intermediate code	Relational algebra	Logical plan
Optimization (IR)	Rewrite rules	Optimized plan
Plan selection	Planner + cost model	Physical plan
Code generation	Produce/consume generator	Bytecode
Optimization (code)	Peephole optimizer	Optimized bytecode
Runtime	Stack virtual machine	Result rows

IV. IMPLEMENTATION

A. Lexer and Parser

The lexer scans the source once and emits keyword, identifier, literal, operator and punctuation tokens, each carrying its line and column. The parser is recursive descent over an EBNF grammar, with one function per rule and operator precedence encoded in the expression rules. The top-level rule is:

```
select_stmt := SELECT [DISTINCT] select_list
               FROM table_ref { join_clause }
               [where] [group_by] [having]
               [order_by] [limit] ;
```

B. Semantic Analysis

Names are resolved against a catalog that acts as the symbol table. The analyzer checks unknown and ambiguous tables and columns, type compatibility, aggregate placement and GROUP BY rules, and reports each problem with a stable error code, a source position and a suggested fix.

C. Intermediate Representation and Rule Optimizer

The AST is lowered to relational algebra with Scan, Filter, Project, Join, Aggregate, Sort, Distinct and Limit operators. Rewrite rules apply predicate pushdown, projection pruning, constant folding, redundant-condition removal and join reordering. Each rule records its reason and estimated impact; on the full-size statistics, pushdown cuts the rows entering the flagship join from about 100,000 to about 20,000.

D. Physical Planning and Cost Model

The planner enumerates join orders, including a Selinger-style dynamic-programming search [1], join algorithms (hash, nested-loop, merge) and access paths (sequential and index scans). A transparent cost model estimates cardinalities from column statistics, interpolating range predicates between column minima and maxima, and the cheapest plan is selected with a written justification.

E. Code Generation

The selected plan is compiled with the produce/consume model [2]. Scans become cursor loops, filters become postfix stack code followed by a conditional jump, and a nested-loop join becomes two nested loops. Pipeline breakers (hash build, sort, aggregation) materialize rows into VM data structures. Listing 1 shows the complete program for a filtered two-table join.

```
0000 SCAN_OPEN     r1, departments
0001 NEXT          r1, @0020
0002 LOAD          r1, name
0003 CONST         'CSE'
0004 EQ
0005 JUMP_IF_FALSE @0001             ; WHERE d.name = 'CSE'
0006 SCAN_OPEN     r2, students
0007 NEXT          r2, @0001
0008 LOAD          r2, cgpa
0009 CONST         8.0
0010 GT
0011 JUMP_IF_FALSE @0007             ; WHERE s.cgpa > 8.0
0012 LOAD          r2, dept_id
0013 LOAD          r1, id
0014 EQ
0015 JUMP_IF_FALSE @0007             ; ON s.dept_id = d.id
0016 LOAD          r2, name
0017 LOAD          r1, name
0018 EMIT          2
0019 JUMP          @0007
0020 HALT
```

Listing 1. Bytecode for `SELECT s.name, d.name FROM students s JOIN departments d ON s.dept_id = d.id WHERE s.cgpa > 8.0 AND d.name = 'CSE'`.

F. Peephole Optimizer

The peephole pass rewrites the label-form code to a fixed point (Table II) and logs each rewrite. Jump threading changes jump targets rather than deleting instructions, so code size can stay constant while executed instructions fall. One classic rule is deliberately excluded: replacing `NOT; JUMP_IF_FALSE` with `JUMP_IF_TRUE` is unsound under SQL's three-valued logic, because `NOT NULL` is `NULL`.

TABLE II
Peephole Rules

Rule	Pattern	Result
Constant folding	CONST 2; CONST 3; ADD	CONST 5
Constant branch	CONST TRUE; JUMP_IF_FALSE L	removed
Jump threading	jump to L where L: JUMP M	jump to M
Jump to next	JUMP L; L:	L:
Dead code	code after JUMP never reached	removed
Store-load fwd.	MAKE_ROW r; LOAD r,0; LOAD r,1	removed

G. Virtual Machine

The VM is a stack machine with 44 opcodes. Named registers hold the row each cursor is positioned on; hash tables, sort buffers, aggregation tables and seen-sets support joins, sorting, grouping and DISTINCT. Comparisons and logical operators follow three-valued semantics. Queries run against a deterministic sample database (336 students, 20 departments, 34 courses, 727 enrollments) scaled so that nested-loop plans finish quickly.

H. Self-Verification and Fuzzing

Every compilation executes both the optimized and the unoptimized program, whose results must match, and compiles and executes every candidate plan, whose rows must equal those of the selected plan; plans that exceed an instruction budget are skipped. A random query generator walks the foreign-key graph to choose joined tables, then adds filters with literals drawn from the sample data and optional grouping, HAVING, ordering, DISTINCT and LIMIT, so every query is valid. It drives the dashboard's Random button and a fuzz test.

I. Dashboard

The React dashboard provides a query editor with examples and the Random button, token and AST views, the original and optimized IR, candidate plans with costs, and a Bytecode & VM page showing the program before and after peephole optimization, each rewrite, the result rows and the plan verification table.

V. RESULTS AND DISCUSSION

Table III reports each example query: code size and executed instructions before/after peephole optimization, the saving, result rows, and agreeing plans out of those executed.

TABLE III
Example Queries

Query	Code	Executed	Saved	Rows	Plans
Department report	99/99	11,291/11,004	2.5%	4	8/8
Top grades in two cities	75/75	6,432/5,813	9.6%	15	9/9
Join + predicate pushdown	25/21	2,393/2,033	15.0%	2	6/6
HAVING on an aggregate	28/28	2,010/2,010	0.0%	20	1/1
LEFT JOIN pushdown boundary	32/29	2,599/2,259	13.1%	2	3/3
Three-way join reordering	54/51	5,527/5,182	6.2%	1	11/11
ORDER BY + LIMIT	23/23	2,390/2,115	11.5%	10	1/1

Correctness. In every example, the optimized and unoptimized programs returned identical results, and every executed candidate plan agreed with the selected plan. Over 200 random queries, 200 compiled and verified, 194 returned rows, and all 936 executed candidate plans agreed (102 exceeded the instruction budget).

Defect found by fuzzing. A query whose WHERE predicate referenced the nullable side of a LEFT JOIN crashed the planner. The optimizer correctly kept that filter above the join, but the planner assumed every filter in a join chain wraps a table scan. The fix plans a filter over a join as its own relation, and a regression test covers it.

Peephole effectiveness. On the flagship query the pass reduced executed instructions from 2,393 to 2,033. Over the random queries it removed 5.5% on average and up to 26.2%. Jump threading contributes most; aggregation-heavy queries gain little because their time is spent in accumulator updates.

Estimated versus executed cost. For Listing 1 the cost model selects a nested-loop plan over sequential scans, yet the cheapest index-scan plan executes 4.0x fewer instructions. The model charges for random page reads that do not exist in an in-memory VM. Executing every plan makes this gap measurable.

Testing. The backend has 233 automated tests covering the front end, every rewrite rule, the planner and cost model, the REST API, code generation against direct Python evaluation, every peephole rule, three-valued logic and 40 fuzzed queries; the frontend has 13 component tests. All pass, and continuous integration runs them on every push.

VI. CONCLUSION AND FUTURE WORK

queryX demonstrates that a complete query compiler, from lexer to executed bytecode, can be small enough to read while exercising every major compiler phase. Executing every candidate plan turns the optimizer's own search space into a test oracle, which together with random query generation found a real defect. The system supports a controlled SQL subset without subqueries, set operations or window functions, interprets bytecode rather than emitting native code, and uses an illustrative cost model. Future work includes histogram statistics, bushy join trees, calibrating the cost model against executed instruction counts, and native code generation.

ACKNOWLEDGMENT

The authors thank their faculty guide for feedback throughout the Compiler Design Lab project reviews.

REFERENCES

[1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie and T. G. Price, “Access path selection in a relational database management system,” in *Proc. ACM SIGMOD*, 1979.

[2] T. Neumann, “Efficiently compiling efficient query plans for modern hardware,” *Proc. VLDB Endowment*, vol. 4, no. 9, 2011.

[3] T. Neumann and M. Freitag, “Umbra: A disk-based system with in-memory performance,” in *Proc. CIDR*, 2020.

[4] G. Graefe, “Volcano: An extensible and parallel query evaluation system,” *IEEE Trans. Knowl. Data Eng.*, vol. 6, no. 1, 1994.

[5] W. M. McKeeman, “Peephole optimization,” *Commun. ACM*, vol. 8, no. 7, 1965.

[6] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[7] W. M. McKeeman, “Differential testing for software,” *Digital Tech. J.*, vol. 10, no. 1, 1998.

[8] M. Rigger and Z. Su, “Testing database engines via pivoted query synthesis,” in *Proc. USENIX OSDI*, 2020.